Time complexity

Friday, April 03, 2026 1:37 AM

What's Polynomial ?
It's a O(n^ k)
K is constant
N is a big num like 1e5

Exponential ?
It's Exponential time complexity refers to functions where the input size appears in the exponent, such as:
`O(c^ n)`

Complexity	Max n
O(1)	∞
O(log n)	∞
O(n)	10^7
O(n log n)	10^6
$O(n^2)$	2000
$O(n^3)$	200
$O(2^n)$	20
O(n!)	10

Note that : 1 sec= 10^8

Polynomial :

- O(1) : **`runs in just one step (constant time)`**

  It means the algorithm runs in a fixed number of steps, independent of the input size.

  Examples:

  - if condition
  - accessing an array element
  - simple arithmetic operations

  O(log n) : **number of steps = how many times we divide n by 2 until we reach 1**

  Examples:

  - Binary Search
  - repeatedly dividing the problem in half

- O(root (n)) :**The algorithm runs up to the square root of n.**

  - `loops run until i * i ≤n`
  - Example: checking divisors or prime checking

- O(n) : **The algorithm goes through all elements once.**

  Example:

  simple loop over an array

- O(n*log n) : **n operations × log n work each**

  Example: efficient sorting (merge sort, quick sort average)

- O(n^2) : **grows by doubling at each step**

  Example: comparing every pair of elements

  O(n^3) :**`triple nested loops`**

  Example: checking all triplet

Exponential :

- O(2^n) : **grows by doubling at each step**

  Example :
  subsets, brute force recursion

- O(n!) : **grows by factorial of n**

Subset:

A **subset** is any possible selection of elements from a given set, including:

- the empty set
- the full set itself

Example:

If you have:

```
S = {1, 2, 3}
```

All possible subsets are:

```
{}
{1}
{2}
{3}
{1,2}
{1,3}
{2,3}
{1,2,3}
```

Important rule:

If a set has **n elements**, the number of subsets is:

2^n

Why?

Each element has 2 choices:

- included
- not included

Factorial:

The **factorial** of a number n (written as n!) is the product of all integers from 1 to n.

Formula:

```
n! = n × (n-1) × (n-2) × ... × 1
```

Examples:

```
5! = 5 × 4 × 3 × 2 × 1 = 120
3! = 3 × 2 × 1 = 6
1! = 1
0! = 1 (important rule)
```

Recursion: is a technique where a function calls itself to solve a problem by breaking it into smaller subproblems.

Every recursive function must have:

1. **Base case** → stops the recursion
2. **Recursive case** → function calls itself

Example 1: Factorial using recursion

We use:

`n! = n × (n-1)!`

C++ code:

```
int factorial(int n) {
    if (n == 0 || n == 1)  // base
case
        return 1;

    return n * factorial(n - 1);
// recursive case
}
```